



ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ

Καρτσιώτης Μιχαήλ ΑΕΜ: 11892

Κατσιμάνης Δημήτριος ΑΕΜ: 11895

Περιεχόμενα

Σύντομη περιγραφή του λογισμικού.....	4
Περιγραφή του περιβάλλοντος ανάπτυξης του προγράμματος.....	4
Λογισμικό	4
Χρήση εργαλείων Τεχνητής Νοημοσύνης.....	5
Άλλες πηγές και εργαλεία.....	5
Οδηγίες μεταγλώττισης του πηγαίου κώδικα.....	5
Απαιτήσεις	5
Εντολές Μεταγλώττισης	5
Μέσω του MAKE.....	5
Απευθείας	6
Οδηγίες εγκατάστασης του εκτελέσιμου αρχείου, των βιβλιοθηκών και των πόρων του προγράμματος.....	7
Για Linux και Mac.....	7
Οδηγίες εκτέλεσης του προγράμματος	7
Εκτελέστε το αρχείο game ή game.exe	7
WINDOWS	7
Linux & Mac Os.....	7
Οδηγίες χειρισμού του λογισμικού	7
Εξέλιξη του έργου.....	8
Φάση 1: Αρχική Σχεδίαση και Επαναπροσδιορισμός Μηχανισμών Κίνησης	8
Φάση 2: Βασική Λειτουργικότητα και Αλγόριθμοι Διαδρομής.....	9
10	
Φάση 3: Διαχείριση NPC και Μετάβαση σε 3D.....	10
Φάση 4: Βελτιστοποίηση Κυκλοφορίας Οχημάτων.....	11
Φάση 5: Σχεδίαση Επιπέδων και Διαχείριση Μνήμης	11
Φάση 6: Σύστημα Βαθμολογίας και Καυσίμων.....	12
Φάση 7: Τελικές Βελτιώσεις και Έλεγχος Ποιότητας.....	13
Προϋποθέσεις και πληρότητα	14
Περιορισμοί τρέχουσας έκδοσης & Προοπτικές εξέλιξης.....	14
Δυσκολίες – Αποτυχημένες ή εγκαταλειμμένες προσεγγίσεις.....	15
Εισαγωγή Ορών- Fractals.....	15
Τι είναι το Perlin Noise:.....	15
Perlin Noise:.....	15
Fractal Χαρακτηριστικά:	15
Λόγοι επιλογής Perlin Noise:	16
Παράρτημα	16

Καταγραφή και περιγραφή όλων των συναρτήσεων που αναπτύχθηκαν στο πρόγραμμα, ανά μεταφραστική μονάδα	16
Layout.c.....	16
Draw.c.....	16
Player_movement.c	18
Gamehandling.c	18
Timer limitation functions.....	19
A* functions. Astar_search.c.....	19
Npc.c.....	19
Cam.c	20
Score handling (All functions in gamehandling.c)	20
Audio.c	21
Αναφορές και Πόροι.....	21
Αναφορές – Άδειες	22
Ευχαριστίες.....	23

Σύντομη περιγραφή του λογισμικού

Το παρόν λογισμικό αποτελεί ηλεκτρονικό παιχνίδι που αναπτύχθηκε με τη χρήση της βιβλιοθήκης raylib.h. Το παιχνίδι δημιουργήθηκε στο πλαίσιο εργασίας στο μάθημα του δομημένου προγραμματισμού κατά το ακαδημαϊκό έτος 2024-2025. Πρόκειται για ένα παιχνίδι διανομέα στο οποίο ο παίκτης καλείται να παραδώσει τον μέγιστο αριθμό παραγγελιών από καταστήματα υγειονομικού περιεχομένου με τη βέλτιστη κατά τις συνθήκες διαδρομή προκειμένου να ελαχιστοποιηθεί η κατανάλωση καυσίμου γεγονός που επιτείνει τη διάρκεια ζωής του παίκτη. Τον παίκτη καταδιώκει αστυνομικό όχημα καθώς διαθέτει ελευθερία παραβίασης των κανόνων του κώδικα οδικής κυκλοφορίας και συχνά αναγκάζεται να τους παραβεί για να παραδώσει την παραγγελία στον λιγότερο δυνατό χρόνο. Πριν την ανάληψη της παραγγελίας εμφανίζεται στην οθόνη λίστα με τις διαθέσιμες αποστολές και τις μέγιστες απολαβές τους. Με την επιλογή μίας αποστολής εμφανίζεται στον χάρτη η παρούσα αμοιβή την οποία ο παίκτης θα λάμβανε αν παρέδιδε την παραγγελία εκείνη τη χρονική στιγμή. Ο παίκτης καλείται λοιπόν όχι μόνον να παραδώσει αλλά και να παραλάβει την παραγγελία από σημεία που επισημαίνονται στον χάρτη του παιχνιδιού. Με την παρέλευση χρόνου όσο ο παίκτης δεν κατορθώνει να επιτύχει την εκπλήρωση της αποστολής του και κατά τη σύγκρουση με άλλα οχήματα οι απολαβές του μειώνονται.

Περιγραφή του περιβάλλοντος ανάπτυξης του προγράμματος

Λογισμικό

Το πρόγραμμα αναπτύχθηκε σε υπολογιστή με λειτουργικό σύστημα WINDOWS 11 με την ενσωμάτωση της βιβλιοθήκης raylib.h στον φάκελο των αρχείων και τη χρήση του προγράμματος επεξεργασίας κώδικα Vs Code και με Codeblocks. Σύμφωνα με τις οδηγίες και τις προδιαγραφές της εργασίας, ο πηγαίος κώδικας του παιχνιδιού διασπάστηκε σε οκτώ μεταφραστικές μονάδες σε 9 αρχεία κώδικα σε γλώσσα C (ένα για κάθε μεταφραστική μονάδα και ένα αρχείο επικεφαλίδων) και ένα αρχείο γραμμένο σε γλώσσα Makefile. Για τη συνεργασία μεταξύ των δημιουργών αξιοποιήθηκε το κατανεμημένο σύστημα λογισμικού ελέγχου εκδόσεων git σε συνδυασμό με το ιστότοπο github ως server για την ανταλλαγή εγγράφων, τον έλεγχο εκδόσεων και την ασφάλεια σε περίπτωση απώλειας δεδομένων. Η μεταγλώττιση του προγράμματος διενεργήθηκε στο τερματικό και οι οδηγίες μετάφρασης προκύπτουν μέσω του makefile, ενός ενσωματωμένου εργαλείου του GNU-GCC για τη διασύνδεση και τη δημιουργία ολοκληρωμένων προγραμμάτων. Για την ανάπτυξη και την τροποποίηση των μοντέλων 3d χρησιμοποιήθηκε το λογισμικό Blender και το λογισμικό 3d slicer για τη διαίρεση των μοντέλων παρόλο που τελικά όλα τα μοντέλα υπέστησαν επεξεργασία μέσω του Blender λόγω του καλύτερου περιβάλλοντος χρήσης. Κάποια 3d μοντέλα αντλήθηκαν από το διαδίκτυο από συγκεκριμένους ιστοτόπους-βιβλιοθήκες και διαθέτουν όλα άδεια χρήσης ελεύθερων δικαιωμάτων με αναφορά στα στοιχεία του δημιουργού(CC 4.0).

Χρήση εργαλείων Τεχνητής Νοημοσύνης

Για την ανάπτυξη του λογισμικού χρησιμοποιήθηκαν και εργαλεία τεχνητής νοημοσύνης παρά το γεγονός ότι δεν αντιγράφηκε κανένα block κώδικα από εργαλεία τεχνητής νοημοσύνης. Πιο συγκεκριμένα αξιοποιήθηκε το εργαλείο GEMINI για την άντληση πληροφοριών σχετικά με την άδεια χρήσης και αποκλειστικά για ερευνητικούς σκοπούς ειδικότερα στα πρώτα στάδια για την οικειοποίηση με τις εντολές και τη βιβλιοθήκη raylib.h και την αναζήτηση διαδρομών και επιλογών ανάπτυξης με αποκλειστικό στόχο τη διευκόλυνση στη λήψη αποφάσεων για την επιλογή της κατεύθυνσης εξέλιξης (όπως παρουσιάζεται σε επόμενο κεφάλαιο).

Άλλες πηγές και εργαλεία

Ιδιαίτερα χρήσιμα υπήρξαν και τα βιβλία Γραμμικός Προγραμματισμός (Σίσκος, 2009) και Μη Γραμμικές Μέθοδοι Βελτιστοποίησης (Γεωργίου & Βασιλείου, 1993) ιδιαίτερα για την ανάπτυξη και την κατανόηση των αλγορίθμων εύρεσης βέλτιστης διαδρομής και την επιλογή του κατάλληλου αλγορίθμου. Ιδιαίτερα πολύτιμη υπήρξε και η βοήθεια του συμφοιτητή YagorNIKON Mykhailo για την κατανόηση των οδηγιών μετάφρασης και ενσωμάτωσης της βιβλιοθήκης raylib και για την ενημέρωση για τις δυνατότητες του make.

Οδηγίες μεταγλώττισης του πηγαίου κώδικα

Απαιτήσεις

Για τη μεταγλώττιση του κώδικα απαιτείται η εγκατάσταση στο λογισμικό του GNU-GCC το οποίο μπορείτε να κατεβάσετε εδώ για windows για λειτουργικά συστήματα που βασίζονται σε Unix ή Linux συνήθως το πακέτο μεταγλωττιστών GCC είναι ήδη εγκατεστημένο, για Mac OS απαιτείται η εγκατάσταση GCC αν και η μεταγλώττιση του προγράμματος μπορεί να γίνει και με άλλα πακέτα μεταγλώττισης με τις κατάλληλες οδηγίες μετάφρασης. Είναι απαραίτητο ιδιαίτερα για λειτουργικά συστήματα windows μετά την εγκατάσταση του GCC να οριστεί η διαδρομή του φακέλου bin ως μεταβλητή περιβάλλοντος συστήματος (Μεταβλητές περιβάλλοντος συστήματος-Μεταβλητές περιβάλλοντος-Προσθήκη διαδρομής-Δημιουργία νέας μεταβλητής και επικόλληση τη διαδρομή). Είναι επίσης βοηθητική η εγκατάσταση του make αν δεν περιλαμβάνεται ήδη στο πακέτο του GCC.

Εντολές Μεταγλώττισης

Μέσω του MAKE

Ανοίξτε νέο τερματικό ως παράθυρο στον φάκελο των αρχείων. Στη συνέχεια πληκτρολογήστε:

WINDOWS

```
mingw32-make
```

LINUX & Mac Os

Πρέπει να τροποποιήσετε το makefile ως εξής:

1. Ανοίξτε το Makefile από το editor της επιλογής σας.
2. Στην αρχή του εγγράφου εντοπίστε τη γραμμή `TARGET := game.exe`.

3. Τροποποιήστε την ως TARGET := game
4. Στο τέλος του εγγράφου εντοπίστε το clean:
5. Διαγράψτε την επόμενη γραμμή(*.o game.exe 2>nul || true) και από την επόμενη γραμμή αφαιρέστε τον χαρακτήρα σχολίου(#)(#rm -f *.o game).
6. Πληκτρολογήστε στο terminal ανοιγμένο στη διαδρομή του φακέλου make
7. Να σημειωθεί εδώ ότι η μαθηματική διασύνδεση της βιβλιοθήκης γίνεται αυτόματα με μακροεντολές(που αντλήθηκαν από τα παραδείγματα του make)

Απευθείας

WINDOWS

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c main.c -o main.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c layout.c -o layout.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c player_movement.c -o player_movement.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c draw.c -o draw.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c gamehandling.c -o gamehandling.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c astar_search.c -o astar_search.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c npc.c -o npc.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c cam.c -o cam.o
```

```
gcc -Wall -Wextra -O2 -std=c17 -I./raylib_2/include -c audio.c -o audio.o
```

```
gcc main.o layout.o player_movement.o draw.o gamehandling.o astar_search.o npc.o cam.o audio.o -L./raylib_2/lib -l:libraylib.a -lopengl32 -lgdi32 -lwinmm -luser32 -lkernel32 -lshell32 -o game.exe
```

LINUX

```
gcc main.o layout.o player_movement.o draw.o gamehandling.o astar_search.o npc.o cam.o audio.o -L./raylib_2/lib -lraylib -lGL -lm -lpthread -ldl -lrt -lX11 -o game
```

Mac Os

```
gcc main.o layout.o player_movement.o draw.o gamehandling.o astar_search.o npc.o cam.o  
audio.o -L./raylib_2/lib -lraylib -framework CoreVideo -framework IOKit -framework Cocoa -  
framework GLUT -framework OpenGL -o game
```

Οδηγίες εγκατάστασης του εκτελέσιμου αρχείου, των βιβλιοθηκών και των πόρων του προγράμματος

Οι πόροι οι βιβλιοθήκες και το εκτελέσιμο αρχείο περιλαμβάνονται στον φάκελο του αρχείου οπότε για λειτουργικό σύστημα windows δεν θα υπάρξει πρόβλημα στη μεταγλώττιση(εφόσον χρησιμοποιείται η .a έκδοσή της raylib)

Για Linux και Mac

Το εκτελέσιμο δεν θα ανοίξει εκτός αν έχετε εγκαταστήσει κάποια επέκταση που να εκτελεί αρχεία .exe. Σε αυτή την περίπτωση συστήνεται να μεταγλωττίσετε τον κώδικα και στη συνέχεια να εκτελέσετε το αρχείο.

Να σημειωθεί εδώ ότι λόγω του περιορισμού στο μέγεθος του αρχείου θα πρέπει να εγκαταστήσετε τη βιβλιοθήκη raylib εξ αρχής από την αντίστοιχη σελίδα της στο github. Στη συνέχεια ο φάκελος της βιβλιοθήκης πρέπει να αντιγραφεί στο εσωτερικό του φακέλου με τον πηγαίο κώδικα του παιχνιδιού.

ΣΗΜΑΝΤΙΚΟ: Ο φάκελος που περιέχει τη raylib πρέπει να ονομαστεί όχι **raylib** αλλά **raylib_2**

Οδηγίες εκτέλεσης του προγράμματος

Εκτελέστε το αρχείο game ή game.exe

WINDOWS

.\game.exe

Linux & Mac Os

./game

Οδηγίες χειρισμού του λογισμικού

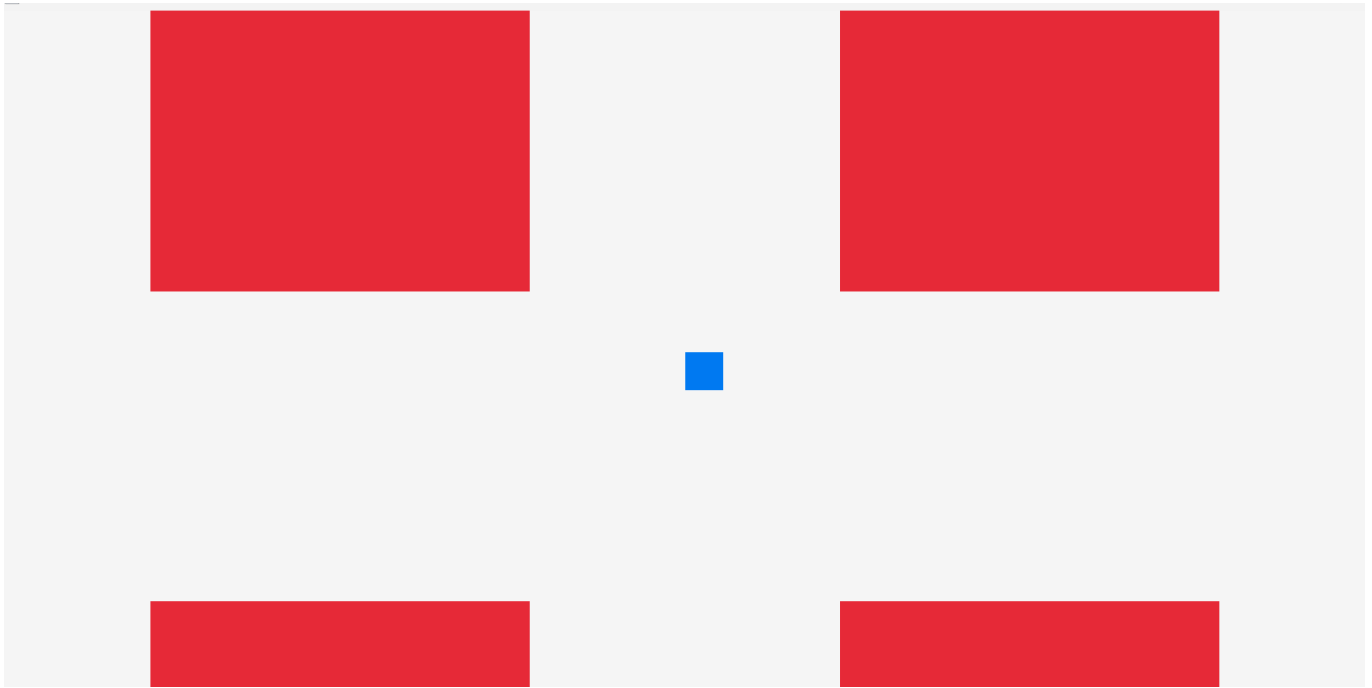
1. Μετά την έναρξη του παιχνιδιού πατώντας space εισέρχεσθε στην κεντρική οθόνη.
2. Από εκεί μπορείτε με τα βελάκια να επιλέξετε επίπεδο.
3. Έχουν δημιουργηθεί 4 επίπεδα αυξανόμενης δυσκολίας στα οποία εισέρχεσθε ανάλογα με τον αριθμό νομισμάτων που έχετε συλλέξει, όπως εμφανίζονται στην οθόνη.
4. Πατώντας Enter εισέρχεσθε στο επιλεγμένο επίπεδο.
5. Η κίνηση του διανομέα γίνεται μέσω των βελών.
6. Για την εναλλαγή αποστολής χρησιμοποιήστε το W ή το S και για την επιλογή αποστολής πατήστε Enter.
7. Μετά την επιλογή αποστολής εμφανίζεται με κίτρινη γραμμή ή βέλτιστη διαδρομή προς το σημείο παραλαβής της παραγγελίας και μετά την παραλαβή η βέλτιστη διαδρομή προς το σημείο παράδοσης

8. Το σκορ που λαμβάνετε με την ολοκλήρωση κάθε αποστολής εμφανίζεται στο κεντρικό πάνω μέρος της οθόνης. Να σημειωθεί ότι κατά την σύγκρουσή σας με άλλα οχήματα και με την παρέλευση χρόνου βαθμοί αφαιρούνται!
9. Κάτω δεξιά στην οθόνη σας εμφανίζεται μία μπάρα με τη διαθέσιμη ποσότητα καυσίμου στη δεξαμενή. Λίγο πριν το απόθεμα στη δεξαμενή εξαντληθεί εμφανίζεται στο χάρτη πρατήριο με τη μορφή δοχείου καυσίμου σε σημείο που επισημαίνεται στον μικρό χάρτη πάνω και δεξιά στην οθόνη με λευκό κύκλο.
10. Λαμβάνοντας το δοχείο καυσίμου που εμφανίζεται παρατείνεται η διάρκεια ζωής σας.
11. Το παιχνίδι τερματίζεται είτε με την σύγκρουσή σας με το αστυνομικό όχημα είτε με την εξάντληση του αποθετηρίου καυσίμων.
12. Πατώντας SPACE, όταν η μπάρα του NITRO είναι γεμάτη αυξάνεται η ταχύτητα του παίκτη.
13. Στόχος: Η συγκέντρωση όσο το δυνατόν περισσότερων χρημάτων που ξεκλειδώνουν επίπεδα και προνόμια.

Εξέλιξη του έργου

Φάση 1: Αρχική Σχεδίαση και Επαναπροσδιορισμός Μηχανισμών Κίνησης

- Στα αρχικά στάδια (Log #2), επιχειρήθηκε η υλοποίηση ενός συστήματος κίνησης του παίκτη σε παγκόσμιο χάρτη (world map) χωρίς τη χρήση της ενσωματωμένης κάμερας της βιβλιοθήκης Raylib. Δοκιμάστηκαν οι εξής προσεγγίσεις:
- Σύστημα συντεταγμένων όπου ο παίκτης κινείται εντός ορίων.
- Σύστημα σταθερού παίκτη στο κέντρο με μετακίνηση του κόσμου.
- Ενδιάμεσο σύστημα κίνησης τόσο του παίκτη όσο και του περιβάλλοντος.
- Αξιολόγηση και Απόφαση: Η προσέγγιση αυτή εγκαταλείφθηκε λόγω υψηλής πολυπλοκότητας στη διαχείριση ορίων, έλλειψης αποδοτικού συστήματος συντεταγμένων και προβλημάτων στη διαχείριση αρχείων/συναρτήσεων. Αποφασίστηκε η επανεκκίνηση της ανάπτυξης με χρήση της συνάρτησης κάμερας της Raylib και εστίαση στην εύρεση βέλτιστης διαδρομής (αλγόριθμοι τύπου Micromouse).

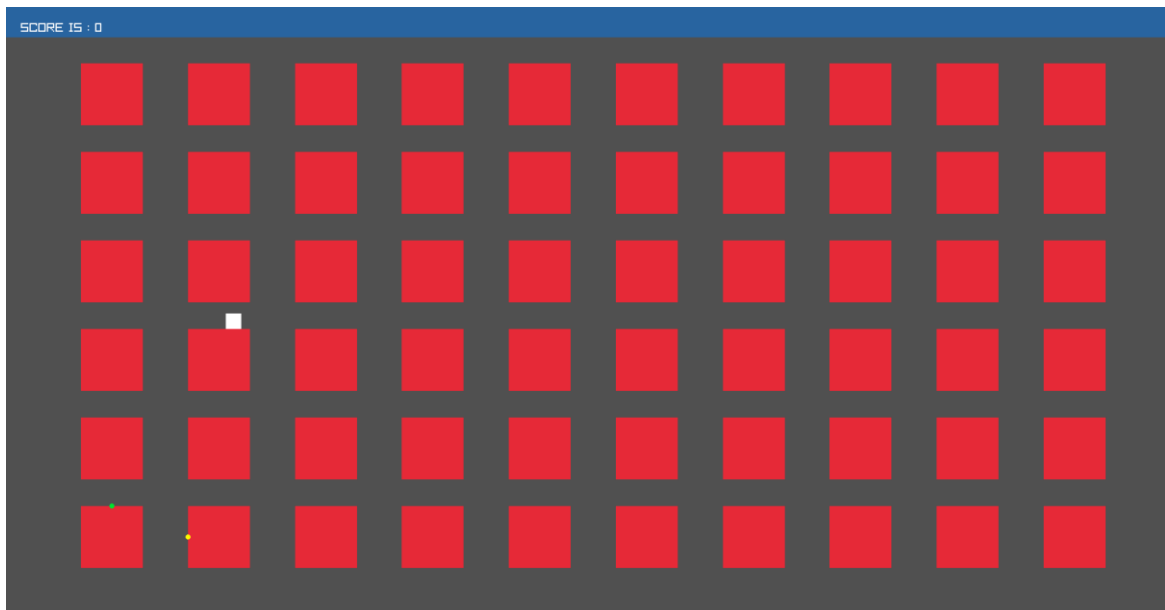


Εικόνα 1: Πρώτη Έκδοση του παιχνιδιού με παγκόσμιο χάρτη

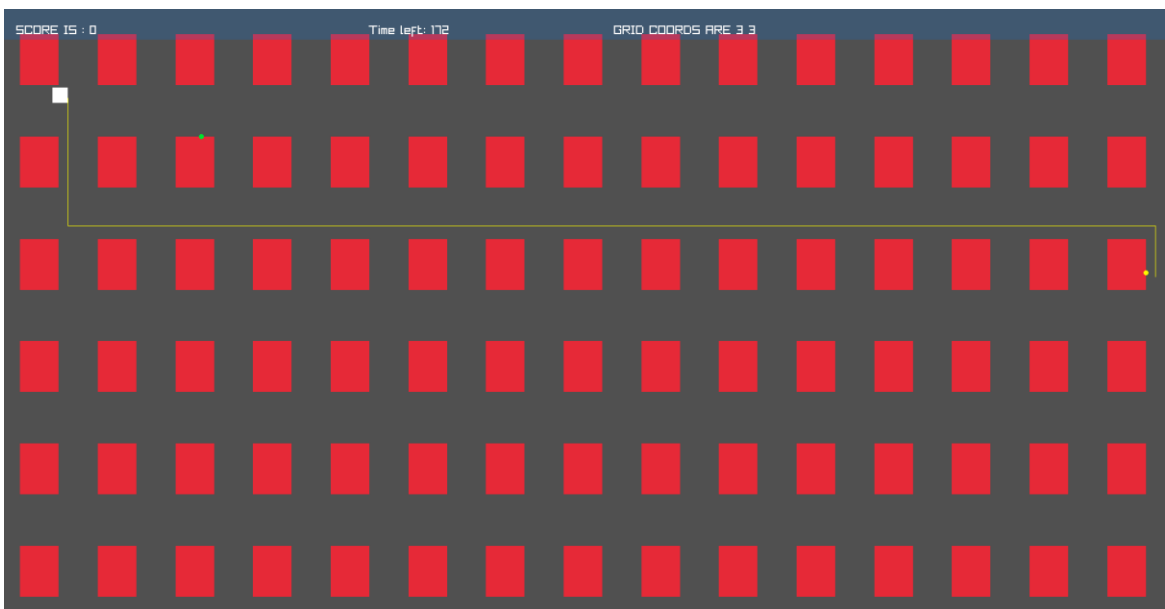
Φάση 2: Βασική Λειτουργικότητα και Αλγόριθμοι Διαδρομής

Στη συνέχεια (Log #4, #5, #6), αναπτύχθηκε η βασική λογική του παιχνιδιού:

- Υλοποίηση συστήματος παραλαβής και παράδοσης παραγγελιών.
- Εισαγωγή λογικής χρονικού περιορισμού.
- Έρευνα για την εφαρμογή 3D γραφικών.
- Ενσωμάτωση Αλγορίθμου A*: Μελετήθηκε και υλοποιήθηκε ο αλγόριθμος εύρεσης μονοπατιού A* (A-Star). Για την εφαρμογή του, δημιουργήθηκε συνάρτηση που χωρίζει τον χάρτη σε πλέγμα (grid) και τροποποιήθηκαν οι συναρτήσεις πινάκων για την αρχικοποίηση του χάρτη βάσει αυτού. Επιτεύχθηκε η σχεδίαση της βέλτιστης διαδρομής στην οθόνη.



Εικόνα 2: Έκδοση παιχνιδιού για δοκιμή αλγορίθμων

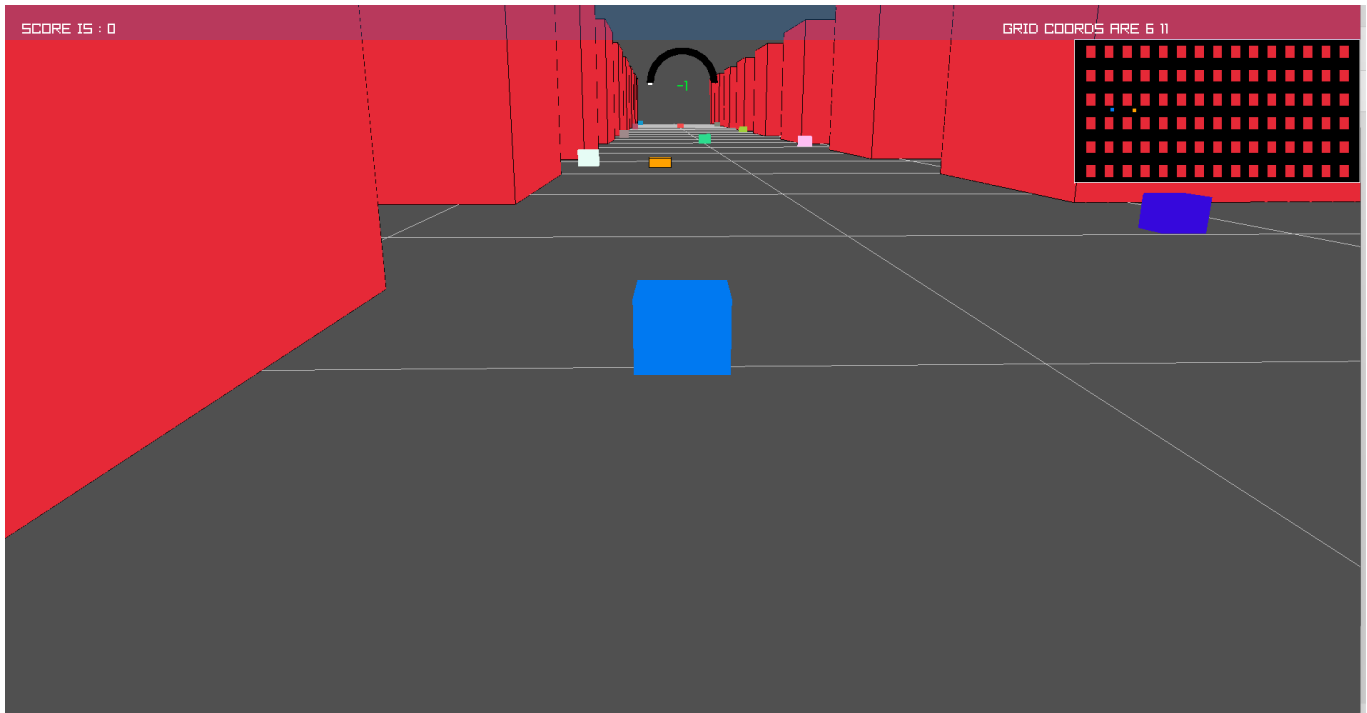


Εικόνα 3: Πρώτη λειτουργική έκδοση με τον A*

Φάση 3: Διαχείριση NPC και Μετάβαση σε 3D

Κατά την προσπάθεια εφαρμογής του αλγορίθμου A* στην κίνηση των χαρακτήρων μη παικτών (NPCs), διαπιστώθηκε υπερβολική πολυπλοκότητα που καθιστούσε τον κώδικα μη διαχειρίσιμο.

- Απόφαση: Εγκατάλειψη του A* για τα NPCs και υλοποίηση απλούστερης λογικής ("Zombie logic").
- Προσθήκες: Υλοποίηση του τρισδιάστατου (3D) περιβάλλοντος, βελτιώσεις στον χειρισμό, προσθήκη ταχύμετρου και 3D εφέ.



Εικόνα 4: Πρώτη έκδοση με κινούμενα οχήματα σε 3 διαστάσεις

Φάση 4: Βελτιστοποίηση Κυκλοφορίας Οχημάτων

Για την κίνηση των οχημάτων (Traffic Logic), εξετάστηκαν τεχνικές "βιομηχανίας" για την προσομοίωση κυκλοφορίας με χαμηλό υπολογιστικό κόστος. Οι προτεινόμενες λύσεις περιλάμβαναν:

1. Εναλλασσόμενη Ροή (Alternating Flow): Κίνηση βάσει του δείκτη δρόμου (μονός/ζυγός) για δημιουργία αίσθησης δρόμων διπλής κατεύθυνσης.
2. Λογική Φρένου (Brake Light): Έλεγχος απόστασης για αποφυγή αλληλοκάλυψης οχημάτων.
3. Καθολικό Σύστημα Φαναριών: Χρήση καθολικού χρονομέτρου για έλεγχο διασταυρώσεων.

Φάση 5: Σχεδίαση Επιπέδων και Διαχείριση Μνήμης

(Log #8, #9) Η ανάπτυξη προχώρησε στη δημιουργία πολλαπλών επιπέδων δυσκολίας:

- Επίπεδο 1: Άδειοι δρόμοι, αργό περιπολικό.
- Επίπεδο 2: Ταχύτερο περιπολικό, παρουσία οχημάτων.
- Επίπεδο 3: Πλήρης κίνηση και αυξημένη δυσκολία.

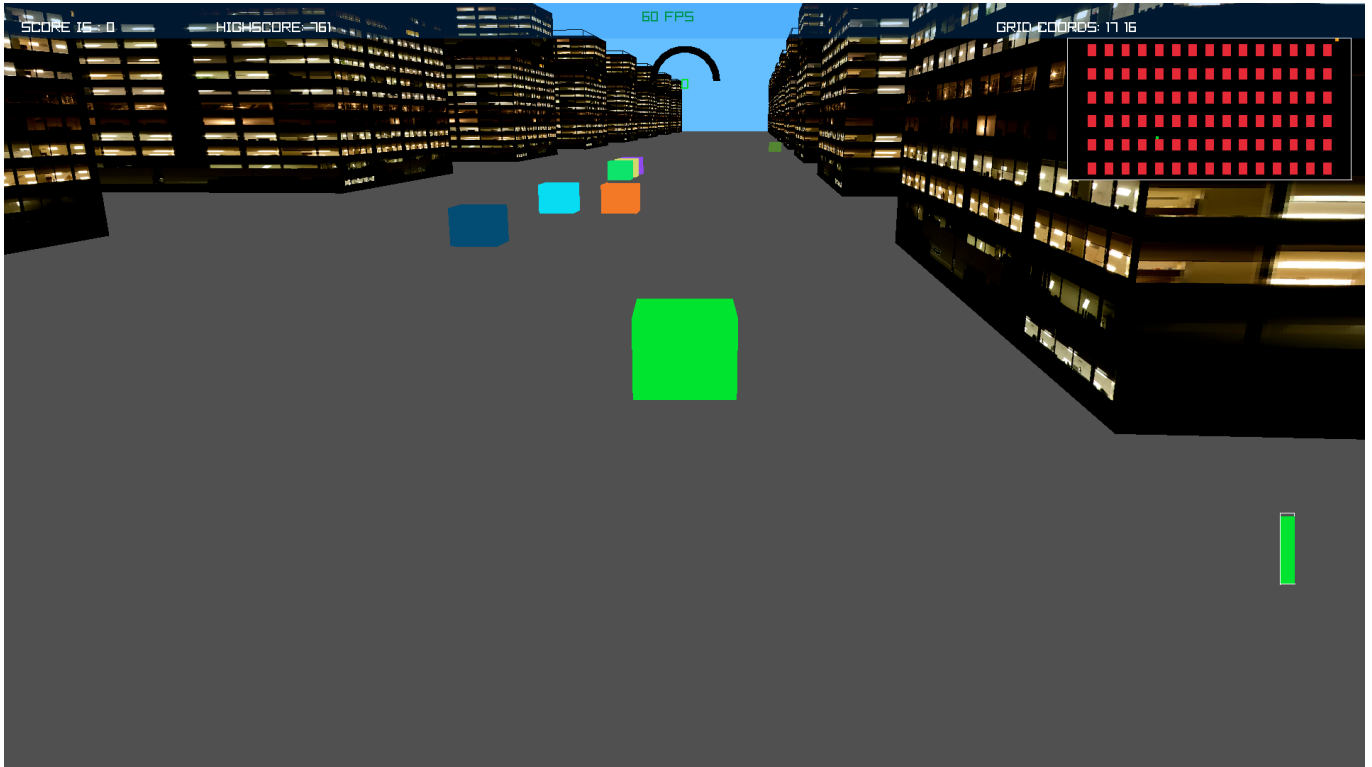
Τεχνική Υλοποίηση:

- Χρήση εξωτερικών μεταβλητών (externs) για παραμέτρους επιπέδων.
- Για την αποφυγή δυναμικής δεσμεύσης μνήμης (calloc) και αλλαγής μεγέθους πινάκων, επιλέχθηκε η χρήση σταθερών μεγάλων πινάκων (fixed arrays) και η αλλαγή της λογικής προσπέλασης.
- Υλοποιήθηκε σύστημα "κλειδώματος" επιπέδων, τα οποία ξεκλειδώνουν βάσει του υψηλότερου σκορ (High Score) στο προηγούμενο επίπεδο.
- Προστέθηκε λειτουργία παράκαμψης (cheat code) για ξεκλείδωμα επιπέδων, καθώς και πλήκτρα πλοήγησης (UP/DOWN Arrow) στην οθόνη προεπισκόπησης.
- Κωδικοποίηση αρχείου High Score στη μορφή: -%LOCKEDLEVEL-%SCORE1-%SCORE2-%SCORE3-.

Φάση 6: Σύστημα Βαθμολογίας και Καυσίμων

(Log #10, #11) Το σύστημα βαθμολόγησης και οι μηχανισμοί τερματισμού αναθεωρήθηκαν:

1. Υπολογισμός Σκορ: Βασίζεται στην απόσταση Manhattan (Pickup to Dropoff) με ποινές για συγκρούσεις και χρόνο.
2. Σύστημα Καυσίμου: Αντικατάσταση του χρονικού ορίου με σύστημα κατανάλωσης βενζίνης. Ο τερματισμός επέρχεται με την εξάντληση καυσίμου.
3. Πρατήρια Καυσίμων: Δημιουργία νέου τύπου αντικειμένου (Gas Station) με συναρτήσεις τυχαίας τοποθέτησης και σχεδίασης στον χάρτη.
4. Προσθήκη Λειτουργιών: Υλοποίηση "Night Mode" (Νυχτερινή λειτουργία).



Εικόνα 5: Πρώτη έκδοση με λειτουργικά 3d γραφικά και βελτιωμένα npc

Φάση 7: Τελικές Βελτιώσεις και Έλεγχος Ποιότητας

Στο τελικό στάδιο (βάσει των τελευταίων καταγραφών και του feedback συνεργασίας), πραγματοποιήθηκαν οι εξής ενέργειες:

- Διορθώσεις Σφαλμάτων (Bug Fixes): Αντιμετώπιση προβλήματος όπου τα NPC κολλούσαν σε συγκεκριμένα σημεία και διόρθωση ένδειξης ταχύτητας (που εμφάνιζε -1/-2 σε στάση).
- Βελτιώσεις Γραφικών/Απόδοσης: Ενεργοποίηση Full Screen Mode, βελτιστοποίηση FPS, βελτίωση χρωματικής παλέτας.
- Λειτουργικότητα: Προσθήκη εξόδου "Game Over", μετατροπή του Score σε Coins, και εφαρμογή αρνητικής βαθμολογίας με την πάροδο του χρόνου.
- Έλεγχος Αρχείων: Επιβεβαίωση μεγέθους συμπιεσμένου φακέλου (2-3 MB) και ορθότητας σχολίων στον κώδικα (headers).
- Τελευταίες Προσθήκες: Εισαγωγή 3D μοντέλων (κτήρια, σκούτερ, NPCs), πεζοδρόμια.
- Δημιουργία και χρήση μουσικής για βελτιστοποιημένη εμπειρία χρήσης.

Προϋποθέσεις και πληρότητα

- Το παιχνίδι έχει ένα χάρτη και λίστα των ενεργών παραγγελιών, με την ελάχιστη αμοιβή της κάθε μίας, η οποία εξαρτάται π.χ. από το κατάστημα και το ύψος της παραγγελίας. Η θέση του διανομέα θα είναι ορατή στο χάρτη και επιλέγοντας τις πληροφορίες μιας παραγγελίας εμφανίζεται η συντομότερη διαδρομή μέχρι το κατάστημα και από εκεί μέχρι το σημείο παράδοσης.
- Ο παίκτης, επιλέγοντας την ανάληψη της παραγγελίας μεταβαίνει στο κατάστημα, παραλαμβάνει την παραγγελία, μεταβαίνει στο σημείο παράδοσης, παραδίδει την παραγγελία. (Εγινε)
- Εκτός από το όχημα του παίκτη, στους δρόμους του χάρτη κινούνται άλλα οχήματα, τα οποία θα πρέπει ο παίκτης να αποφύγει τη σύγκρουση μαζί τους.
- Οι μεταβάσεις δεν είναι απαραίτητο να γίνουν ακολουθώντας τη βέλτιστη διαδρομή και η παράδοση πρέπει να ολοκληρώνεται συντομότερο δυνατό, με τον παίκτη να επιλέγει εναλλακτικές διαδρομές.
- Οι δρόμοι έχουν συγκεκριμένες κανονικές κατευθύνσεις κίνησης των οχημάτων, με τα άλλα οχήματα να τις ακολουθούν, και τον διανομέα να έχει την ελευθερία παραβίασης τους.
- Κάθε φορά που εκτελείται μία έγκαιρη παράδοση, ο διανομέας θα λαμβάνει την ελάχιστη αμοιβή του, ενώ, αν την παραδώσει ταχύτερα θα λάβει επιπλέον επιβράβευση.
- Αν καθυστερήσει πέραν ενός ορίου, η αμοιβή του μειώνεται.
- Σε κάθε επόμενο επίπεδο, η δυσκολία αυξάνεται αυξάνοντας το πλήθος των άλλων οχημάτων, αυξάνοντας το απαιτούμενο σύνολο των αμοιβών και αυξάνοντας τον αριθμό των κινούμενων οχημάτων.
- Τα επίπεδα δεν είναι χρονικά περιορισμένα με τη λογική πεπερασμένης ποσότητας καυσίμου και ανάγκης ανατροφοδοσίας σε πρατήριο.
- Επιλογής και αγοράς διαφορετικών οχημάτων, π.χ. ηλεκτροκίνητων ή ποδηλάτων, με διαφορετικά χαρακτηριστικά κόστους απόκτησης, συντήρησης, τροφοδοσίας, ταχύτητας κ.λπ.(πρέπει να γίνει)

Περιορισμοί τρέχουσας έκδοσης & Προοπτικές εξέλιξης

Η τρέχουσα έκδοση του παιχνιδιού έχει και αδυναμίες και πιο συγκεκριμένα:

1. Η διάταξη του παιχνιδιού είναι ορθογώνια και σε μορφή τετραγώνων. Αυτό συμβαίνει καθώς η διατήρηση της απλότητας ήταν ο πιο σημαντικός στόχος κατά την ανάπτυξη του παιχνιδιού. Επιλέξαμε έτσι να αφιερώσουμε σημαντικό χρόνο και πόρους στη δημιουργία της 3d έκδοσης του παιχνιδιού με αποτέλεσμα να μην μπορέσουμε να παραθέσουμε με ακρίβεια τα χαρακτηριστικά μιας πραγματικής πόλης με λοξούς δρόμους, φωτεινούς σηματοδότες και περίπλοκα γραφικά. Έτσι ο αριθμός των επιπέδων είναι

περιορισμένος. Μια πολύ ενδιαφέρουσα μελλοντική εξέλιξη θα ήταν η δημιουργία τυχαίων επιπέδων με μία συνάρτηση αλλά κάτι τέτοιο δεν ήταν εφικτό δεδομένων των χρονικών συνθηκών καθώς θα απαιτούσε αλλαγή μεγάλου μέρους της υπάρχουσας αρχιτεκτονικής.

2. Τα ηrc για λόγους απλούστευσης έχουν απλή κίνηση(ευθύγραμμη) και εμφανίζονται και εξαφανίζονται σχετικά απότομα στο τέλος του παιχνιδιού. Μια προηγμένη εκδοχή θα ήταν η αντιμετάθεση των αυτοκινήτων από τους κάθετους στους οριζόντιους δρόμους και η δημιουργία αλγορίθμων που θα επιτρέπουν τη στροφή τους.
3. Fractal logic για τη δημιουργία ορών και χάρτη.
4. Φυσικά η άδεια δικαιωμάτων ανοιχτού κώδικα επιτρέπει μελλοντικές προσθήκες και συνεργασίες για την αναβάθμιση της ποιότητας του παιχνιδιού.

Δυσκολίες – Αποτυχημένες ή εγκαταλειμμένες προσεγγίσεις

Η μεγαλύτερη δυσκολία της παρούσας εργασίας αναμφίβολα υπήρξε η διατήρηση ενός αξιοπρεπούς ρυθμού ανανέωσης οθόνης. Οι δυσκολία έγκειται στο γεγονός ότι τα 3d μοντέλα τυπώνονται πολλαπλώς γεγονός που απαιτεί χρόνο και εξαντλεί τους διαθέσιμους πόρους επεξεργασίας. Έτσι ο μεγαλύτερος φόρτος και πόνος εργασίας ήταν η έρευνα και η ανίχνευση του κώδικα για παραλείψεις αβλεψίες και γενικότερα για βελτιστοποιήσεις. Λόγω του μεγάλου μεγέθους της εργασίας η ανίχνευση αυτών των αβλεψιών υπήρξε ιδιαίτερα χρονοβόρα. Έτσι δοκιμάστηκαν τα εξής:

1. Instanced rendering για τα στατικά γραφικά
2. Εκτύπωση και ενημέρωση ηrc και χρήση συναρτήσεων A^* μόνο σε περιορισμένα χρονικά διαστήματα
3. Επιλεκτική εκτύπωση πλησίον γραφικών

Εισαγωγή Ορών- Fractals

Τι είναι το Perlin Noise:

Το Perlin Noise fractals ουσιαστικά σημαίνει η χρήση του αλγορίθμου Perlin Noise (μιας τεχνικής δημιουργίας ομαλής, ψευδο-τυχαίας υφής) σε πολλαπλά επίπεδα (octaves) με διαφορετικές συχνότητες και πλάτη, ώστε να προκύψουν πολύπλοκες, «φράκταλ» όπως μορφές που μοιάζουν με φυσικά φαινόμενα (βουνά, σύννεφα), συνδυάζοντας πολλές στρώσεις θορύβου για μεγαλύτερη λεπτομέρεια.

Perlin Noise:

- Αναπτύχθηκε από τον [Ken Perlin](#), είναι ένας αλγόριθμος που παράγει ομαλό, «βαθμωτό» θόρυβο (gradient noise).
- Χρησιμοποιείται για τη δημιουργία υφής, εδάφους, και άλλων ψευδο-τυχαίων μοτίβων.

Fractal Χαρακτηριστικά:

- Ο απλός Perlin θόρυβος είναι ομαλός, αλλά η κλασματομορφή του προκύπτει όταν συνδυάζονται πολλές "οκτάβες" (octaves).

- Κάθε οκτάβα είναι ένα επίπεδο θορύβου με διπλάσια συχνότητα (πιο λεπτομερές) και μειωμένο πλάτος (amplitude) από την προηγούμενη.
- Αυτός ο συνδυασμός ονομάζεται [fractal noise](#) και προσδίδει στο τελικό αποτέλεσμα λεπτομέρεια σε πολλαπλές κλίμακες, όπως ακριβώς τα φράκταλ, κάνοντάς το να φαίνεται "φυσικό".

ΠΗΓΗ: <https://www.cs.montana.edu/courses/spring2005/525/students/Thiesen1.pdf>

Λόγοι επιλογής Perlin Noise:

1. Ευκολία εισαγωγής χωρίς περίπλοκα μαθηματικά με συνάρτηση raylib.
2. Δυνατότητα εξοικονόμησης χώρου και υπολογιστικής ισχύος.

Η ιδέα για τη χρήση φρακταλικών μορφών προήλθε από διάλεξη του κ. Κεχαγιά για τις χρονοσειρές, τις ακολουθίες χάους και τις φρακταλικές μορφές και διαστάσεις (housedorff dimensions) και τις εφαρμογές τους.

Η εφαρμογή έγινε με την αξιοποίηση των παραδειγμάτων της raylib.h.

Να σημειωθεί επίσης ότι λόγω έλλειψης χρόνου οι συναρτήσεις και αλγόριθμοι δημιουργίας 2d εικόνων και μετάφρασης τους σε μοντέλα αντλήθηκαν από το παράδειγμα:

https://www.raylib.com/examples/models/loader.html?name=models_heightmap_rendering

Επίσης αναζητήθηκε βοήθεια για την εξομάλυνση των ορών από εργαλείο τεχνητής νοημοσύνης για την εύρεση αλγορίθμου εξομάλυνσης με την εισαγωγή παραμέτρου γραμμικής απόσβεσης του ύψους γύρω από ορισμένη ακτίνα του μοντέλου.

Έτσι μετά από ορισμένες ακόμα βελτιστοποιήσεις έγινε η ενσωμάτωση των ψευδοτυχαίων ορών μέσω του Perlin Noise.

Παράρτημα

Καταγραφή και περιγραφή όλων των συναρτήσεων που αναπτύχθηκαν στο πρόγραμμα, ανά μεταφραστική μονάδα.

Layout.c

```
void Initialize_Map(Rectangle (*map)[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X]);
```

//(In layout.c) Initialize the map of the square blocks that will constitute the road. Parameters Map array that we created.

Draw.c

```
void draw_pickup_and_dropoff(Vector2 PICKUP, Vector2 DROPOFF);
```

// Draws small circles around dropoff and pickup locations. Parameters are PICKUP and DROPOFF the array of the PICKUP and DROPOFF points.

```
void draw_pickup_and_dropoff3D(Vector2 PICKUP, Vector2 DROPOFF);
```

// Draws cones circles around dropoff and pickup locations. Parameters are PICKUP and DROPOFF the array of the PICKUP and DROPOFF points.

```
void DrawRectangles(Rectangle map[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X]);
```

// (In draw.c) Draw the array of Rectangles Initialized as map in the main void function to create a map. Parameters is the map array that we have to initialize draw and control the walls.

```
void Draw_and_update_score_window(int sucessful_deliveries, int HIGH, ScreenStatus GameScreen);
```

//(In draw.c) Draws a score window. Parameters int successful_deliveries(current score), enum screen_status(current screen)

```
void draw_astar_results(best_possible_path A_STAR_RESULT);
```

// Draws A* results in 2D. Parameter(Astar type) is the result of astar_search(we have a custom type)

```
void draw_astar_results3D(best_possible_path A_STAR_RESULT);
```

// Draws A* results in 3D. Parameter(Astar type) is the result of astar_search(we have a custom type).

```
void draw_npc(NPC chaser);
```

// Draws an NPC. Parameter npc type(the type of the police car)

```
void drawspeed(void);
```

//(in draw.c). Draws a speedometer.

```
void draw_fuel_bar(void);
```

//(In draw.c)Draws the remaining fuel in the deposit.

```
void draw_grid(void);
```

//(In draw.c)Draws the grid of the big map in world-map coordinates. Note that this function does not draw the lines of the coordinates of the grid[i][j] but the outside sides of the rectangles that represent a 2D division of the map plane.

```
void DrawCubes(Rectangle map[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X], Vector2 playerPos);
```

// Draws the cubes for the 3D version. Parameters are the array of rectangles.

```
void draw_npc3D(NPC chaser);
```

// Draws the NPC in 3d. Parameters the npc type that contains all the info about the police car.

```
void draw_cars(Vector3 camera3d);
```

// Draws all cars. Parameters the camera's or the players position to ensure effective and selective drawing.

```
void draw_mission_score(int selected_mission_index);
```

// Draws the score that is going to be awarded if no more points are deducted. Parameter is the intex of the sected mission.

```
void Draw_list_of_deliveries(Delivery_Location PICKUP[NUM_OF_ITEMS_ON_LIST], Delivery_Location DROPOFF[NUM_OF_ITEMS_ON_LIST]);
```

// Draws the list of possible deliveries. Parameters PICKUP and DROPOFF typedefed structures that contain the info for pickup and dropoff.

void draw_nitro_bar(int t);

// DRAWS Nitro bar. Parameters int t is a counter variable that is used to calculate nitro in tank.

Color choseRandomColour(void);

// Returns a random color from rand via a decoding method.

Color LerpColor(Color start, Color end, float factor);

//(in draw.c)Fades a color. Parameters the colors that are going to lerp and the factor by the amount of f.

Player_movement.c

bool check_for_collisions(Rectangle Player, Rectangle
map[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X]);

// (In player_movement.c) Check for collisions between the player and the grid objects. Return 0 if no collision
1 if we collided. Parameters Map array and player's position.

void keep_in_boundaries(Vector2 *pos);

//(In player_movement)Checks and modifies the pos.x and pos.y if player is out of the window. Parameters
pointer to player position.

float delta_move(void); //(In player_movement.c) Calculates according to user input the required movement.
Returns the movement in an axis(separate calls for x and Y needed).

Gamehandling.c

Delivery_Location initialize_pickup_location(Rectangle[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X]);

// Sets pickup location. Returns a typedefed struct that is of type delivery location. Parameters the rectangle
array of the building blocks.

Delivery_Location initialize_dropoff_location(Rectangle
map[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X], Vector2 PICKUP);

// Sets dropoff location. Returns a typedefed struct that is of type delivery location. Parameters the rectangle
array of the building blocks.

void check_and_apply_mission(Delivery_Location *PICKUP, Delivery_Location
*DROPOFF);

// Sets all the parameters for current mission. Parameters is the pickup and dropoff points array.

void init_PICKUP_and_DROPOFF(Delivery_Location *PICKUP, Delivery_Location
*DROPOFF);

```
// Initializes PICKUP and DROPOFF location. Parameters are pointers to pickup and dropoff arrays.
```

Timer limitation functions

```
void burn_fuel(void);
```

```
//(In gamehandling.c)Decreases the fuel amount.
```

A* functions. Astar_search.c

```
void initGrid(void);
```

```
// Initializes the grid that will be used for finding the best path
```

```
void CreateWalls(void);
```

```
// Sets the walls where they need to be.
```

```
double calculateH(int x, int y, int destX, int destY);
```

```
// Calculates heuristic function. This function is used to calculate the A* path. ( $h(n) + g(n) = f(n)$ ). Returns the Heuristic value in float. Parameters coordinates of destination and of the start point of the function.
```

```
bool isValid(int x, int y);
```

```
// Check validity of coordinates given. Returns false if invalid true if valid.
```

```
best_possible_path aStarSearch(int startX, int startY, int destX, int destY);
```

```
// Output the A* results. Return type the struct type for the a_star algorithm. Parametrs same as the previous function.
```

```
grid_coordinates RealToGrid(Vector2 pos);
```

```
//(In astar_search.c) converts real map coordinates (type real coordinates) to grid coordinates for a_star_search. Remember pos is the center position not the top-left.
```

```
Vector2 GridToReal(int gridX, int gridY);
```

```
//(In astar_search.c) converts grid coordinates to real map ones. Inverse function and parameters of the previous.
```

Npc.c

```
void updateNPC(NPC *chaser, Vector2 player_pos, Rectangle  
map[NUM_OF_RECTANGLES_Y][NUM_OF_RECTANGLES_X]);
```

```
//(In npc.c) Checks conditions and recalculates path if needed. Parameter is the rectangle array.
```

```
int check_if_caught(Vector2 playerpos, NPC npc);
```

```
//(In npc.c)Checks if they come in contact. Parameters are the player position and the npc position.
```

```
void update_npc_cars(void);
```

```
//(In npc.c)Controls npc car movement and specifically APPEARANCE - DISAPPEARANCE - POSITION - COLOUR  
- START AND END POSITION.  
  
void init_cars(void);  
  
// Initializes the array of cars  
  
int check_for_car_crashes(Rectangle Player);  
  
// Checks for collisions with the npc cars. Parameters the player hit-box and return value is 0 if not caught and  
1 if caught.  
  
void adjust_speedx(int i, int j);  
  
//(In npc.c)This function sets the speed of the following cars to the speed of the the slower moving first car.  
Parameters are the index values of cars.  
  
void adjust_speedy(int i, int j);  
  
//(In npc.c)This function sets the speed of the following cars to the speed of the the slower moving first car.  
Parameters are the index values of the cars.  
  
void adress_traffic(void);  
  
//(in npc.c)This function creates a vertical and a horizontal go for the cars.  
  
Cam.c  
void TurnCam(Camera3D *camera3d, Vector2 pos); // Turns 3D cam.  
  
// Screen logic and level logic. Parameters pointer to camera 3d and player position.  
  
void set_game_parameters(ScreenStatus *GameScreen, NPC *npc);  
  
//(in layout.c) Sets all the parameters before a game level starts. Parameters are the current screen (see enum)  
and the npc typedefed struct.  
  
Score handling (All functions in gamehandling.c)  
void set_score_for_current_mission(Vector2 pos, Vector2 PICKUP, Vector2 DROPOFF, int i);  
  
// Calculates the score for a given mission and assigns it to the global external variable  
(score_for_current_mission). Parameters are player position pickup and dropoff typedefed struct.  
  
void deduce_score_for_mission(int n, int selected_mission_index);  
  
// Deduces score for current mission when called in main(for time deduction and punishment when hitting a  
car or when doing something illegal). Parameters are the n number of score reduction and the mission index.  
  
// Gas station parameters  
  
Gas_Station refuel_station(void);
```

//(In gamehandling.c) This is the function that is responsible for setting the gas station when the fuel low enough at a random position. Returns a gas_station typedefed struct.

```
void print_refuel_station(Gas_Station SET_STATION);
```

//(In gamehandling.c) Prints the station if visible. Parameters is the gas station typedefed type.

```
void check_for_refuel(Gas_Station *STATION, Vector2 pos);
```

//(In gamehandling.c) Checks if the player is near to a gas station and refuels. Parameters are the gas station struct and the player pos vector.

Audio.c

```
void InitGameAudio();
```

```
// Initialize the audio
```

```
void UpdateGameAudio(void);
```

```
// Update it every frame
```

```
void StopGameAudio(void);
```

```
// Stop the music
```

```
void CloseGameAudio(void);
```

```
// Unload the audio model
```

```
void SwitchTrack(int track);
```

// Switches audio tracks. Parameters the n number of track(1 for background to drift music change, 0 for inverse)

Αναφορές και Πόροι

Για την ανάπτυξη του λογισμικού και των παραπάνω συναρτήσεων χρησιμοποιήθηκαν οι εξής πόροι:

Βιβλιοθήκη Raylib: Η βασική βιβλιοθήκη για τη διαχείριση γραφικών, παραθύρων και ήχου (raylib.h).

Τεκμηρίωση: [raylib.com](https://www.raylib.com)

Αλγόριθμος A (A-Star):* Χρησιμοποιήθηκε για την εύρεση βέλτιστων διαδρομών στο `astar_search.c`. Η υλοποίηση βασίστηκε στη θεωρία γράφων και ευρετικών αλγορίθμων.

Άδεια Χρήσης: Ο κώδικας διανέμεται υπό την GNU General Public License (GPL), όπως αναφέρεται στις επικεφαλίδες των αρχείων.

Μαθηματικά Μοντέλα: Χρήση διανυσματικών υπολογισμών για την κίνηση της κάμερας και του παίκτη (Vector2, Vector3, Lerp) όπως παρέχονται από τη raymath.h.

Αναφορές – Άδειες

Ο κώδικας συνοδεύεται από άδεια δικαιωμάτων χρήσης GPL-V3.

Τα 3d μοντέλα αντλήθηκαν από τον ιστότοπο <https://free3d.com/>, υπέστησαν επεξεργασία και συνοδεύονται την άδεια δικαιωμάτων του ιστοτόπου. (<https://free3d.com/help/en/articles/9937612-terms-conditions>)

Bus:

<https://free3d.com/3d-model/bus-setia-negara-low-poly-557005.html>

MOTOR BIKE:

<https://free3d.com/3d-model/simple-motorcycle-2-981288.html>

CARS:

<https://free3d.com/3d-model/low-poly-car-92833.html>

<https://free3d.com/3d-model/ford-pick-up-577519.html>

TANK:

<https://modelvault3d.com.pl/elementor/fuel-canister-jerrycan-3d-model/>

BUILDING:

<https://free3d.com/3d-model/buildinghouse-04-40137.html>

Αποποίηση ευθύνης:

Τονίζεται ότι βάσει της άδειας χρήσης των ιστοτόπων από τους οποίους αντλήθηκαν τα μοντέλα, απαγορεύεται ρητά οποιαδήποτε δημοσίευση των αρχείων των 3d μοντέλων χωρίς άδεια χρήσης από τους δημιουργούς τους. Η συμπερίληψή τους στον πηγαίο κώδικα της εργασίας πραγματοποιήθηκε αποκλειστικά για τις ανάγκες της εξέτασης της και συνεπώς οι προαναφερθέντες δεν φέρουν καμία ποινική ευθύνη σε περίπτωση αναδιανομής ή δημοσίευσής τους. Η χρήση του κώδικα για οποιαδήποτε σκοπό είναι ωστόσο επιτρεπτή.

Η μουσική είναι CC ελεύθερη και αντλήθηκε από τους ιστοτόπους:

<https://opengameart.org/content/matts-creative-commons-music>

<https://pixabay.com/music/search/drifting/>

Εικονίδιο αρχικής οθόνης: Oleksandr Yashnyi

Οι υπόλοιπες εικόνες δημιουργήθηκαν από εμάς και συνοδεύονται με άδεια δικαιωμάτων CC4.0.

Ευχαριστίες

Για την διεξαγωγή της παρούσας εργασίας θα θέλαμε να ευχαριστήσουμε τα μέλη των οικογενειών μας και τους συμμαθητές μας για τη στήριξη και ιδιαίτερα για την πολύτιμη ανατροφοδότηση που παρείχαν και συνέβαλαν σημαντικά στην αναβάθμιση της ποιότητας του παιχνιδιού.